Virtual290 — Demo Requirements & Pluggable Model Stack

Companion to `PLAN.md`. Two parts:

- **Part A** — what you need (software + hardware) to demo one graduate theorem on the board with live interruption.
- **Part B** — the model abstraction layer, so offline compilation and real-time presentation can each use a different model, local or remote, swappable by config.

Part A — The Demo

A.1 Demo scope

One theorem, one proof, ~10 minutes, live interruptible. Not a full paper. Deliberately skip the ingestion pipeline — hand-author the semantic graph for one theorem in JSON. Ingestion is engineering you already know how to do; the demo exists to answer *does this feel like a lecture, and does interruption work?*

Pick a theorem with:

- a proof that fills **2–3 boards** (not 1, not 8) — enough to exercise the erase policy,
- **one mechanical computation** SymPy can verify, so you can demo the correctness gate,
- **one hypothesis worth dropping** ("what if we drop completeness?") so you have a natural deep-tier question.

Banach fixed-point, Arzelà–Ascoli, or Baire category all fit. Better: a lemma from a paper of yours — the demo lands much harder when you can vouch for every line.

Demo targets

| | Target | |—|—| | Lecture length | 8–12 min | | Board ops | 40–60 | | Panels | 3 | | Scripted interruptions | 3 (one per tier — see below) | | Time to first audio after user stops speaking | $<$ 800 ms | | LaTeX validity | 100% |

The three interruptions to rehearse — each demos a different tier:

1. *"Wait, what's rho_n again?"* → **reflex tier**, pure notation-table lookup, ~200 ms, hand points at where it was defined.
2. *"Why does that step follow?"* → **fast tier**, retrieves the justification from the graph, ~1.2 s, underlines the relevant subexpression.
3. *"What if the space isn't complete?"* → **deep tier**, reasoning model, 10–30 s, avatar turns to the board and writes a counterexample attempt as scratch work while thinking.

If all three land cleanly, the system is real.

A.2 Software requirements

Core runtime

| Component | Version / choice | Notes | |—|—|—| | Node | 20+ | Frontend build | | Python | 3.11+ | Backend, verification | | Browser | Chrome/Edge 120+ | WebRTC + Web Audio + `AudioWorklet`. Safari's WebRTC is workable but fussier. |

Frontend

- TypeScript + React + Vite
- **KaTeX** — typesetting. (MathJax if you need `\begin{align}` edge cases; KaTeX is ~10× faster and enough for the demo.)
- Board rendering: SVG (easier hit-testing for `point`/`annotate` targets) or `<canvas>` (better for chalk texture/particles). Start SVG.
- Web Audio API for chalk SFX + audio playback with interrupt
- Rive or Lottie for the avatar rig — **or defer entirely for the demo** and ship a pointer hand only

Backend

- FastAPI + `uvicorn`
- WebSocket relay between browser and model providers (never put API keys in the browser)
- **SymPy** — mechanical-step verification
- **Pydantic** — BIL schema definition; gives you JSON Schema for free, which feeds constrained decoding (§B.4)

Ingestion (skip for demo, needed for M1)

- `latexpand`, `de-macro` (TeX Live)
- `plasTeX` (primary) and/or `LaTeXML` (fallback)
- TeX Live full install ≈ 5 GB

The component everyone forgets: math-aware TTS preprocessing. `\int_0^1 f(x)\,dx` must become *"the integral from zero to one of f of x, d x"* before it reaches any TTS. Raw LaTeX into a TTS produces gibberish and instantly kills the illusion. Build a rule table for the ~150 common patterns (integrals, sums, norms, subscripts, Greek, operators, quantifiers) with an LLM fallback for the rest. **Budget 2–3 days.** It is a bigger deal than it sounds and there is no good off-the-shelf option.

Local model serving (only if going local — see A.3)

- `llama.cpp` (GBNF grammars, best single-user latency) or **vLLM** (XGrammar constrained decoding built in, better if you'll batch)

- Ollama if you want the easy path — supports JSON Schema via its `format` parameter
- ASR: NVIDIA **Parakeet TDT** (fastest streaming) or `distil-whisper` / `faster-whisper`
- VAD: **Silero VAD v5** — 85–100 ms endpointing
- TTS: **Kokoro-82M** (tiny, fast, good) or Qwen3-TTS
- Convenient shortcut: the Hugging Face open speech-to-speech pipeline (July 2026) wires all of the above together **and exposes an OpenAI Realtime-compatible WebSocket API** — meaning you write your client once and swap local remote without touching frontend code. See §B.2; this is the single most useful integration fact for your pluggability requirement.

A.3 Hardware requirements

Three tiers. **Do the demo on Tier 0.**

Tier 0 — Demo laptop, everything remote *start here*

| Item | Requirement | |—|—| | Machine | Any laptop from the last ~4 years. M-series MacBook, or x86 + 16 GB RAM. | | GPU | None. KaTeX + SVG on integrated graphics is nothing. | | **Headphones** | **Required, not optional.** | | Mic | Headset or USB condenser. Built-in laptop mic is acceptable but noticeably worse for endpointing. | | Network | Wired or solid 5 GHz Wi-Fi. Latency jitter is what will embarrass you on stage, not bandwidth. |

> **The headphones point is load-bearing.** With open speakers, the model's own voice hits the mic, the VAD reads it as user speech, and the lecturer interrupts itself in a loop. Browser AEC helps but is not reliable enough to demo on. Headphones make the problem disappear. Learn this here rather than in front of an audience.

Cost: ~$1–5 to compile the theorem (one-time, cached) + ~$0.06–0.11/min live on `gpt-realtime-2.1`, or $0.02–0.05/min on the mini with prompt caching working. A 10-minute demo runs well under a dollar.

Tier 1 — Hybrid: local real-time, remote compile *recommended lab setup*

Local small model handles reflex + fast tiers; compilation and deep reasoning go to a frontier API.

| Option | Spec | Notes | |—|—|—| | **Apple** | M4 Pro / M4 Max, **36–48 GB** unified | Quiet, one box, MLX ecosystem is good now. ~30–40 W. | | **NVIDIA** | RTX 4090 (24 GB) or **5090 (32 GB)** | 5090's 1,792 GB/s bandwidth gives 2–3× the tokens/sec on models that fit. |

Minimum viable: **16 GB unified memory, or 8 GB VRAM** runs a 7–9B model at 4-bit alongside Whisper-base. That's the floor, and it's tight once TTS is also resident. 24–32 GB is comfortable.

VRAM budget for the live path:

```
 8-9B LLM @ 4-bit ..... ~5.5 GB Parakeet TDT / distil-whisper . ~1.5 GB Kokoro-82M
TTS ..... ~0.5 GB KV cache + overhead ..... ~2 GB      ~9.5 GB
```

Tier 2 — Fully local, including compilation

Only worth it if the papers are confidential or you want zero marginal cost.

| Option | Spec | Trade-off | |—|—|—| | Mac Studio M4 Max/Ultra, **128 GB+** | Runs a 70B-class compiler locally | Only sub-$5k desktop that fits 70B+ unquantized — but ~⅓ the bandwidth of a 5090 | | 2× RTX 5090 (64 GB) | Fast on everything that fits | Power, noise, NVLink-less model splitting |

The clean way to think about the buy decision:

> **The real-time path is bandwidth-bound → favors NVIDIA. The compile path is capacity-bound → favors Apple unified memory.**

If you must pick one box for both, an M4 Max with 128 GB is the reasonable compromise — the compile path is where local hardware actually saves you money, and 400ms-vs-250ms on the fast tier is imperceptible to a listener.

Honest recommendation: compilation quality is the thing that determines whether the lectures are any good, and frontier models are meaningfully better at it than anything you can run locally. Keep compilation remote until you have evidence a local model is good enough. Go local on the *real-time* tier first — it's the easier win, and it's where per-minute cost accumulates.

A.4 Latency budget

Local pipeline (RTX 4090/5090 or M4 Max, 8B @ 4-bit):

```
 Silero VAD v5 endpointing ........ 85–100 ms Parakeet TDT streaming ASR ....... 100–
200 ms LLM time-to-first-token .......... 80–200 ms Kokoro TTS first chunk ...........
100–200 ms Output buffer ..................... ~50 ms         Time to first audio .........
400–750 ms
```

Remote native speech-to-speech (`gpt-realtime-2.1`):

```
 Network RTT ...................... 30–80 ms Server VAD + model ............... 300–500
ms         Time to first audio .............. 350–600 ms
```

Both comfortably clear the 2-second target — **local is genuinely competitive here**, which is the case for Tier 1.

Deep tier is 5–60 s by design and is covered by the thinking animation (`PLAN.md` §3.7). The rule that makes this work: **the reflex tier always emits audio within 500 ms**, even if only *"hm — let me think about that."* Perceived latency tracks time-to-first-sound, not time-to-answer.

A.5 Demo checklist

- [ ] BIL schema + validator (`pydantic`, exports JSON Schema)

Board renderer: sequential glyph reveal, chalk texture, 3 panels, erase animation

Hand-authored semantic graph for the chosen theorem (notation table included)

Compiled board script, reviewed line by line by you

LaTeX speech preprocessor

Realtime voice loop with barge-in

Three interruption paths wired and rehearsed

Thinking state with scratch-corner streaming

Pointer hand (avatar optional)

Offline fallback recording of the full lecture — if the network dies mid-demo, play the video

---

Part B — Pluggable Model Stack

Your instinct is right and it should be a first-class design constraint from day one, not a refactor later. The two paths have opposite requirements:

| | **Compiler** (offline) | **Presenter** (real-time) | |—|—|—| | Optimize for | Quality | Latency | | Context | 50–150k tokens (whole paper + LaTeX) | **2–4k tokens** | | Structured output | Essential, complex | Essential, simple | | Reasoning depth | Deep | Shallow (except deep tier) | | Cost model | One-time per paper, cacheable | Per minute, recurring | | Tolerable latency | Minutes | Milliseconds | | **Implication** | Frontier remote model | **Small local model is viable** |

That last row is only true because of a design choice worth stating explicitly:

> **Never send the paper to the real-time model.** Send the current beat plus retrieved graph nodes — 2–4k tokens. This is what makes an 8B local model viable at all, *and* what makes prompt caching effective on the remote path (the difference between $0.06/min and $0.46/min).

B.1 Roles, not models

Configure by **role**. Code never names a model.

| Role | Phase | Needs | Latency | Default remote | Local option | |—|—|—|—|—|—| | `compiler` | offline | 128k+ ctx, structured output, strong reasoning | minutes OK | Opus / GPT-5 / Gemini Pro | Qwen3.6-27B+, 70B-class | | `critic` | offline | independent review of compiler output | minutes OK | **a different family than `compiler`** | — | | `reflex` | online | *none* — KB lookup, no generation | $<$ 300 ms | — | — | | `fast` | online | tool calling, 4k ctx | $<$ 2 s | `gpt-realtime-2.1-mini`, Haiku | Qwen3.5-9B, Gemma 4 8B | | `deep` | online | reasoning traces | 5–60 s | o-series, Opus w/ thinking | — | | `asr` | online | streaming | $<$ 300 ms | Deepgram, or native S2S | Parakeet TDT, distil-whisper | | `tts` | online | streaming, low first-chunk | $<$ 300 ms | Cartesia, ElevenLabs | Kokoro-82M, Qwen3-TTS |

Two notes:

- `reflex` **deliberately has no model.** ”What’s rho_n?” is a dictionary lookup against the notation table. Resisting the urge to put an LLM here is most of how you hit 200 ms.
- `critic` **must be a different model family from** `compiler`**.** Self-review by the same model is close to worthless — it reproduces its own blind spots. Compile with Opus, critique with GPT-5, or vice versa.

B.2 Two abstraction boundaries make this nearly free

You don't need a general "LLM abstraction framework." You need exactly two interfaces, both of which already exist:

1. Voice: the OpenAI Realtime WebSocket protocol. It has become the de facto interface — and critically, the open local stacks (including the HF speech-to-speech pipeline) expose a **Realtime-compatible WebSocket API**. So: write the frontend against the Realtime event protocol once, and switch between OpenAI, a compatible vendor, and a fully local server by changing a URL. No frontend changes. This is the highest-leverage decision in Part B.

2. Board: the BIL schema. Every model — frontier or 8B local — must emit JSON validating against your BIL schema. Validation happens on your side, so a weak model produces *rejected output*, never a corrupted board. Model swapping stops being scary because the blast radius is bounded.

A pleasant side effect: **BIL validity rate becomes a free, automatic model-quality benchmark.** Swap in a candidate model, compile your 10 regression papers, compare validity rate, SymPy pass rate, and referential-integrity violations. You get model selection as a measurement rather than a vibe.

B.3 Provider interface

Keep it small. Resist agent frameworks.

"'python @dataclass(frozen=True) class Capabilities: context*window: int structured*output: Literal["native", "grammar", "prompt*only"] tool*calling: bool reasoning*trace: bool # can we stream thinking scratch corner? native*audio: bool supports*caching: bool cost*per*mtok*in: float cost*per*mtok_out float

class ModelProvider(Protocol): caps: Capabilities async def stream(self, msgs, *, schema=None, tools=None) -> AsyncIterator[Chunk]: ... async def complete(self, msgs, *, schema=None, tools=None) -> Response: ...

Implementations: AnthropicProvider, OpenAIProvider, GoogleProvider,

VLLMProvider, LlamaCppProvider, OllamaProvider

“‘

Config, not code:

“‘yaml

config/models.yaml

profiles: demo: # everything remote — Tier 0 compiler: {provider: anthropic, model: claude-opus-5, thinking: true} critic: {provider: openai, model: gpt-5} fast: {provider: openai, model: gpt-realtime-2.1-mini, native_audio: true} deep: {provider: anthropic, model: claude-opus-5, thinking: true}

hybrid: # Tier 1 — local real-time, remote compile compiler: {provider: anthropic, model: claude-opus-5, thinking: true} critic: {provider: google, model: gemini-pro} fast: {provider: vllm, model: Qwen3.5-9B-Instruct, endpoint: http://localhost:8000, structured_output: grammar} deep: {provider: anthropic, model: claude-opus-5, thinking: true} asr: {provider: local, model: parakeet-tdt} tts: {provider: local, model: kokoro-82m}

airgapped: # Tier 2 compiler: {provider: vllm, model: Qwen3.6-32B, structured_output: grammar} critic: {provider: vllm, model: gemma-4-27b} # different family — intentional fast: {provider: llamacpp, model: gemma-4-8b-q4, grammar: bil.gbnf} deep: {provider: vllm, model: Qwen3.6-32B, thinking: true}

fallback: fast: [local, remote*mini, remote*large] # degrade gracefully, log every switch budget: max*usd*per*pa
*10.00 max*usd*per*session: 2.00 “‘

B.4 Capability negotiation — the part that actually bites

Small local models cannot reliably produce complex structured output by prompting alone. **Constrained decoding is what makes them safe**, and it’s non-negotiable for the `fast` role:

| `structured_output` | Mechanism | Use when | |—|—|—| | `native` | Provider’s JSON-Schema / tool-calling mode | Frontier APIs, Gemma 4, Qwen 3.6 via Ollama `format` | | `grammar` | **XGrammar** (vLLM/SGLang/TensorRT-LLM default) or **GBNF** (llama.cpp), compiled from your BIL JSON Schema | Any local model | | `prompt_only` | Prompt + parse + retry | Last resort. Cap retries at 2, then fall back. |

Compile `bil_schema.json` → GBNF/XGrammar grammar as a build step. With grammar constraints on, even small models emit syntactically perfect BIL on the first pass — so the failure mode of a weak local model degrades from *”corrupt board”* to *”valid but pedagogically mediocre board op.”* That is a completely different risk class, and it’s what makes local models usable here at all.

Other negotiations the router must handle:

- *No `native_audio`* → chain VAD → ASR → LLM → TTS instead of speech-to-speech.
- *No `reasoning_trace`* → thinking state falls back to a generic animation instead of streamed scratch work.

- *No `supports_caching`* → real-time cost estimate jumps ~4×; warn at session start.
- *Small `context_window`* → tighten graph retrieval to top-k nodes.

B.5 Cost reference

| Path | Cost | |—|—| | Compile one paper, frontier model | ~$1–5, **one-time and cached** | | Live session, `gpt-realtime-2.1` | $0.06–0.11/min with caching ($0.18–0.46 without) | | Live session, `gpt-realtime-2.1-mini` | $0.02–0.05/min | | Live session, local | electricity | | 30-min lecture, remote | ~$2–3 (~$1 on mini) |

Two things follow. First, **caching is not an optimization, it's the difference between viable and not** — a 4× swing. Design the real-time prompt so the stable prefix (system prompt + lecture plan summary) never changes mid-session. Second, at these rates the economic case for local hardware is weak for personal use and strong only if you're running many hours of sessions — another reason to start on Tier 0.

B.6 Build order for pluggability

1. Define `ModelProvider + Capabilities` **before writing any model call.** Retrofitting this is painful; doing it upfront costs an afternoon.
2. Implement one remote provider. Ship the demo.
3. Add the config profile system with `demo` only.
4. Add a local provider (vLLM or llama.cpp) for `fast`. Now you have two, which is what actually proves the abstraction — one provider always looks abstract and never is.
5. Add grammar-constrained decoding for local structured output.
6. Add fallback chains + per-role cost/latency telemetry.
7. Run the 10-paper regression suite across profiles. Now model choice is an empirical question.

Sources

- OpenAI Realtime API pricing 2026 · measured session data · cost & latency traps
- Local speech-to-speech assistant stack · Whisper + local LLM + Kokoro · local voice assistant 2026
- Structured output from local LLMs · structured output libraries ranked · reliable JSON from local LLMs
- RTX 5090 vs Mac Studio M4 Max for local AI · Mac vs NVIDIA for local LLMs · 2026 local LLM hardware guide
- Best small language models 2026 · open-source SLMs